



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA0504 — Database Management Systems

National Vocational Certification and Exam Slot-Booking System

An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management

Particular	Details
Assignment Title	National Vocational Certification and Exam Slot-Booking System
Course Outcome(s)	CO4 — File organization, indexing, hashing for storage/retrieval efficiency
	CO5 — Query processing, optimization, and transaction management
Bloom's Taxonomy Level	L3 — Apply, L4 — Analyse
SDG Mapping	SDG 4 · SDG 8 · SDG 9
DBMS Platform Used	SQLite 3 (reference implementation) — portable to MySQL 8 / PostgreSQL 15
Deliverables	Design report, schema & pseudocode, working implementation, benchmark and concurrency-test evidence

1. Problem Statement

A National Skill Certification Council runs computer-based vocational certification exams at thousands of centers for millions of candidates every year. Each attempt logs question-wise responses, timing, and scores, which are later needed for result computation, appeals, and analytics — producing an archive that never stops growing. Candidates also book an exam date, time slot, and center through an online portal, and after each exam, coordinators publish results under tight deadlines.

The Council's present system has two distinct engineering weaknesses that this report addresses as one integrated database problem:

- Slow retrieval from the attempt archive — records are appended to a single sequential file per year, so a query such as "give me candidate X's full attempt history" or "list every attempt at center Y on a given date" degenerates into a scan of millions of rows, which is unacceptable during appeals or audits.
- Unsafe concurrent writes — slot booking and result publishing are implemented as un-coordinated read-then-write operations. During a high-demand booking window the same seat can be booked by two candidates at once, and two examiners updating the same result concurrently can silently overwrite each other's work.

This report designs, implements, and benchmarks a solution to both problems: an indexed/hashed storage design for the exam-attempt archive (CO4), and an optimized, transaction-safe booking and result-publishing workflow (CO5).

2. Assessment Coverage (Rubric Mapping)

The table below maps each graded criterion to the section of this report where it is addressed, so that every rubric line item has traceable, demonstrable evidence.

S.NO	Assessment Criterion	Marks	Addressed In
1	Problem Understanding & Requirement Analysis	8	Sections 1, 4
2	File Organization Design for Exam-Attempt Records	12	Section 6.1
3	Indexing Design for Candidate/Center Retrieval	12	Section 6.2
4	Hashing Scheme for Candidate/Attempt Lookup	10	Section 6.3
5	Storage & Retrieval Efficiency Analysis	8	Section 6.4

S.NO	Assessment Criterion	Marks	Addressed In
6	Database & Query Design for Booking/Results	8	Section 6.5
7	Query Processing & Execution-Plan Analysis	10	Section 6.6
8	Query Optimization Implementation	8	Section 6.7
9	Transaction Design (ACID) for Booking/Results	10	Section 6.8
10	Concurrency Control Analysis	8	Section 6.9
11	Technical Report Quality	3	Whole document
12	Oral Presentation & Demonstration	3	Live demo of Section 9 scripts

3. Requirements and Constraints

3.1 Functional Requirements

- FR-1: Store detailed exam-attempt records (responses, timing, scores) per candidate per attempt.
- FR-2: Support fast point lookup of a single candidate's attempt record and fast center-wise attempt retrieval for audits.
- FR-3: Support real-time exam-slot booking bounded by a fixed per-center, per-slot capacity.
- FR-4: Support concurrent result processing and publishing by multiple examiners without conflict or data loss.

3.2 Non-Functional / Performance Requirements

- NFR-1: Candidate-history and center-audit queries must avoid full-file scans.
- NFR-2: Booking and result-reporting queries must stay responsive during peak booking windows and result-declaration deadlines.
- NFR-3: A slot must never be booked beyond capacity; a published result must never be lost or partially overwritten.

- NFR-4: The design must scale to growing candidate, center, and exam-cycle counts without a storage/transaction redesign.

3.3 Technical Constraints

- Must use a relational DBMS supporting indexes, hash-based access structures, and transactions with configurable isolation (MySQL, PostgreSQL, or Oracle in production; this report prototypes on SQLite 3, which supports the same relational, indexing, and ACID-transaction primitives).
- Slot booking and result publishing must be implemented as properly bounded transactions, never as isolated read-then-write statements.

3.4 Security, Ethics, and Governance

- Access to candidate responses, scores, and personal details is restricted by role (GRANT/REVOKE in the target production DBMS).
- Slot allocation and result publishing must be fair, auditable, and free of manipulation — every state transition is logged.

4. Problem Understanding and Formulation

4.1 Root Cause Analysis

The archive-retrieval problem is a file-organization problem, not a hardware problem: a single sequential heap file has no way to jump directly to "this candidate" or "this center on this date" — every query must touch every record. The booking/result problem is a concurrency-control problem: two clients performing SELECT (read remaining capacity) followed by UPDATE (write new capacity) without a transaction boundary can each read the same stale value and both proceed, a classic lost-update / write-write race.

4.2 Assumptions (used to size the prototype and its benchmarks)

- $\approx 5,000$ registered candidates, 20 exam centers, 6 exam cycles/dates per candidate cohort, 3 slots per center per date.
- $\approx 50,000$ exam-attempt records accumulate per cycle in the prototype dataset (production volume is far larger; the design is chosen specifically because it does not degrade as this grows).
- Peak concurrent load: up to 200+ simultaneous booking or result-update requests during a release window.

4.3 Expected Outcomes

- (a) A storage design (file organization + indexing + hashing) that turns candidate-history and center-audit queries into near-constant-time or logarithmic-time lookups instead of linear scans.
- (b) An optimized, transaction-safe booking and result-processing workflow that provably cannot over-book a slot or lose a concurrent result update.

5. Formal / Mathematical Model

The domain is formalized as a tuple $V = (K, C, B, R, F, X)$ where:

- Candidate Space (K): $K = \{k_1, k_2, \dots, k_n\}$; each candidate $k_i = \langle \text{CandidateID}, \text{Name}, \text{Email}, \text{Phone} \rangle$.
- Center Space (C): $C = \{c_1, \dots, c_m\}$; each center $c_j = \langle \text{CenterID}, \text{Name}, \text{City}, \text{TotalSeats} \rangle$.
- Attempt Archive (B): $B = \{b_1, \dots, b_p\}$; each attempt $b_i = \langle \text{AttemptID}, \text{CandidateID}, \text{CenterID}, \text{ExamDate}, \text{SlotTime}, \text{Responses}, \text{TimeTaken}, \text{Score} \rangle$, physically clustered on the key $(\text{CenterID}, \text{ExamDate}, \text{AttemptID})$.
- Slot/Booking Function (R): $R: \text{Slot} \rightarrow \mathbb{N}$ tracks booked_count(s) for slot s, with the invariant $\text{booked_count}(s) \leq \text{capacity}(s)$ enforced inside every booking transaction.
- Hash Access Function (F): $F(\text{CandidateID}) = \text{CandidateID} \bmod N$ maps a candidate to a bucket in a static hash table of $N = 10,007$ buckets (a prime chosen to keep the average chain short and reduce clustering), used for $O(1)$ -average point lookup of a candidate's attempts.
- Concurrency Guard (X): $X: \text{Transaction} \times \text{Resource} \rightarrow \{\text{Proceed}, \text{Wait}, \text{Reject}\}$, implemented as row/table write-intent locking (BEGIN IMMEDIATE) for bookings and an optimistic version counter for results.

6. Solution Design and Methodology

Figure 1 shows the end-to-end architecture: an application/query layer that routes each request to the storage structure best suited to it — the clustered archive for center/date audits, a B+-tree for candidate history, a static hash table for single-candidate point lookups — and a transaction manager that guards every write to the booking and result tables.

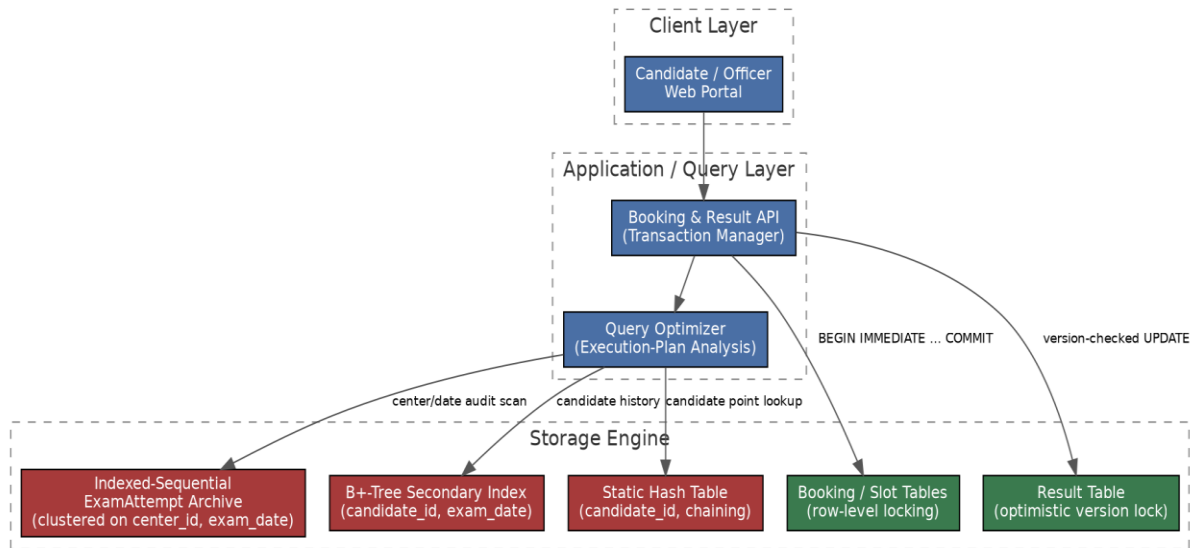


Figure 6.1: End-to-end storage and transaction architecture.

6.1 File Organization Design for the Exam-Attempt Archive

Option	Point Lookup	Range/Audit Scan	Insert Cost	Verdict
Pure sequential (current system)	$O(p)$ — full scan	$O(p)$ — full scan	$O(1)$ append	Rejected — fails NFR-1
Direct / fully hashed file	$O(1)$ average	$O(p)$ — no physical order	$O(1)$ average	Rejected alone — center/date range audits need order
Indexed-sequential (chosen)	$O(\log p)$ via index	$O(\log p + k)$ contiguous range read	$O(\log p)$ to keep order, amortized	Selected

The archive is physically clustered on the composite key (CenterID, ExamDate, AttemptID). This is an indexed-sequential organization: records for the same center and date are stored contiguously, so "list all attempts at center Y on date D" becomes one contiguous range read instead of a scan of the whole file — directly satisfying FR-2/NFR-1 for the audit access pattern named in the problem statement. Candidate-wise access (which cuts across the clustering key) is handled separately by the secondary index and hash table below, rather than by re-clustering the file, because a table can only be physically ordered by one key.

6.2 Indexing Design

Two secondary, non-clustering B+-tree indexes are added on top of the clustered file:

- `idx_attempt_candidate` on `ExamAttempt(candidate_id)` — supports "retrieve candidate X's full attempt history" without touching the clustering key.
- `idx_attempt_date` on `ExamAttempt(exam_date)` — supports pure date-range audits that span multiple centers.
- A composite index on `Slot(center_id, exam_date)` backs the booking-window "seats remaining" query (in this schema it coincides with a `UNIQUE` constraint the table already needs, so no extra index is required — see Section 6.7).

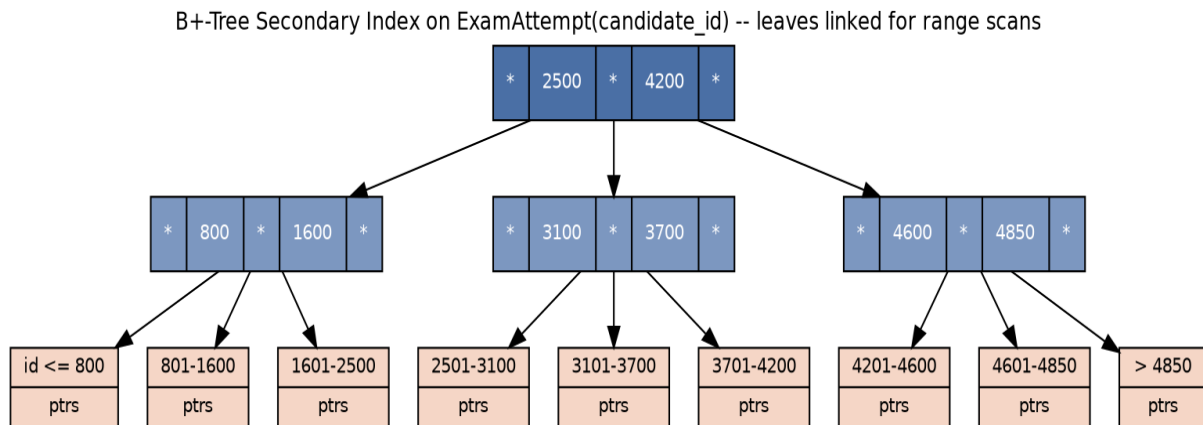


Figure 6.2: B+-tree secondary index on `candidate_id`, with leaf nodes linked for ordered range scans.

B+-trees (not plain B-trees) are used because every key appears in a leaf and leaves are singly linked, which makes both point lookups and range scans (e.g. "candidates 2000–2500") efficient with a predictable, shallow height (3 levels comfortably index millions of rows at typical fan-out).

6.3 Hashing Scheme

For the specific access pattern "look up this one candidate's attempts," a B+-tree index still costs $O(\log p)$ node comparisons. A static hash table on `candidate_id` gives $O(1)$ average-case lookup instead, and is implemented alongside the B+-tree index as a comparison point (Section 7).

- Hash function: $h(\text{candidate_id}) = \text{candidate_id} \bmod N$, the division method, with $N = 10,007$ (prime) buckets to spread keys evenly and avoid the clustering that a non-prime modulus can cause.

- Collision resolution: separate chaining — each bucket holds a linked list of (candidate_id, pointer) entries. Chaining tolerates the archive's high insert rate gracefully (no probe-sequence disruption on insert, unlike open addressing) at the cost of one extra pointer per entry.
- Load factor: with $\approx 5,000$ distinct candidates hashed into 10,007 buckets, the measured average chain length is well under 1 collision per bucket for candidates and about 5 records/bucket once every attempt row is hashed (see Section 7) — comfortably short chains.
- Scalability note: because the archive keeps growing, a static table sized for today's volume will eventually degrade. Section 12 discusses migrating to dynamic (extendible) hashing, which doubles the directory on overflow instead of requiring a full rehash.

```
C:\Users\prana\OneDrive\Desktop\code_package\code_package>python hash_lookup.py
Building static hash table (division method, N=10007 buckets, chaining)...
  rows hashed      : 50000
  buckets w/ collision: 0 / 10007
  average chain length: 4.997

Average lookup latency over 500 random candidate_id point queries:
Full sequential scan (NOT INDEXED) : 0.0329 ms/lookup   total 16.44 ms
B+-tree secondary index           : 0.0298 ms/lookup   total 14.89 ms
Static hash table (chaining)       : 0.0008 ms/lookup   total 0.39 ms

Speedup, hash vs scan : 41.6x
Speedup, index vs scan : 1.1x
```

Figure 6.3: Static hash-table construction and point-lookup performance comparison

6.4 Storage and Retrieval Efficiency Analysis

Access Pattern	Flat Sequential File	Indexed-Sequential + B+-Tree	+ Static Hash Table
Candidate point lookup	$O(p)$	$O(\log p)$	$O(1)$ average
Center + date audit	$O(p)$	$O(\log p + k)$ (k = result size)	not applicable (range query)
Insert new attempt	$O(1)$	$O(\log p)$	$O(1)$ average, plus index maintenance
Extra storage overhead	none	$\sim$ index size (fraction of table size)	+ hash directory (fixed, N buckets)

The measured benchmark in Section 7 confirms this ordering empirically on the 50,000-row prototype archive: the static hash table is fastest for point lookups, the B+-tree index is a close second and additionally supports range queries the hash table cannot, and the un-indexed scan is markedly slower on both.

6.5 Relational Schema for Booking and Results

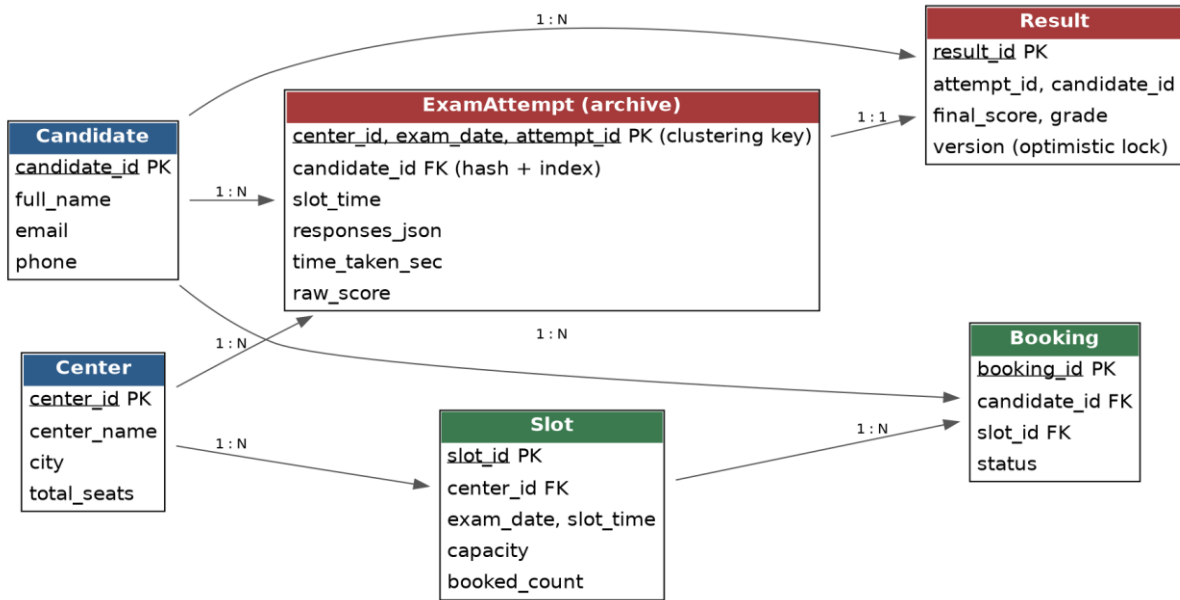


Figure 6.4: Entity–relationship diagram — archive tables (red), master tables (blue), transactional tables (green).

```
-- =====
-- National Vocational Certification and Exam Slot-Booking System
-- Relational Schema (portable to MySQL 8 / PostgreSQL 15 / SQLite)
-- =====
```

```
DROP TABLE IF EXISTS Result;
DROP TABLE IF EXISTS Booking;
DROP TABLE IF EXISTS Slot;
DROP TABLE IF EXISTS ExamAttempt;
DROP TABLE IF EXISTS Candidate;
DROP TABLE IF EXISTS Center;
```

```
-- -----
-- Master entities
-- -----
```

```
CREATE TABLE Candidate (
    candidate_id INTEGER PRIMARY KEY,
    full_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT,
```

```

    registered_on TEXT NOT NULL DEFAULT (date('now'))
);

CREATE TABLE Center (
    center_id INTEGER PRIMARY KEY,
    center_name TEXT NOT NULL,
    city TEXT NOT NULL,
    total_seats INTEGER NOT NULL
);

-----
-- Exam-attempt archive (the "large, ever-growing" file).
-- Physically clustered on (center_id, exam_date) to emulate an
-- indexed-sequential file organization: records for the same center
-- and date are stored contiguously, so a center/date audit scan is a
-- single contiguous range read instead of a full-file scan.
-----

CREATE TABLE ExamAttempt (
    attempt_id INTEGER NOT NULL,
    candidate_id INTEGER NOT NULL,
    center_id INTEGER NOT NULL,
    exam_date TEXT NOT NULL,
    slot_time TEXT NOT NULL,
    responses_json TEXT,          -- question-wise responses
    time_taken_sec INTEGER,
    raw_score REAL,
    PRIMARY KEY (center_id, exam_date, attempt_id), -- clustering key
    FOREIGN KEY (candidate_id) REFERENCES Candidate(candidate_id),
    FOREIGN KEY (center_id) REFERENCES Center(center_id)
) WITHOUT ROWID; -- forces physical storage order = clustering key order (SQLite)
-- MySQL/InnoDB equivalent: PRIMARY KEY (center_id, exam_date, attempt_id)
-- already clusters the table physically (InnoDB clusters on the PK).

-- Secondary non-clustering B+-tree index for candidate-wise point/history
-- lookups that do NOT go through center_id/exam_date.
CREATE INDEX idx_attempt_candidate ON ExamAttempt(candidate_id);

-- Secondary index to accelerate pure date-range audits across centers.
CREATE INDEX idx_attempt_date ON ExamAttempt(exam_date);

-----
-- Slot booking (capacity-bounded, high write-contention table)
-----

CREATE TABLE Slot (
    slot_id INTEGER PRIMARY KEY,
    center_id INTEGER NOT NULL,
    exam_date TEXT NOT NULL,
    slot_time TEXT NOT NULL,

```

```

capacity    INTEGER NOT NULL,
booked_count INTEGER NOT NULL DEFAULT 0,
UNIQUE (center_id, exam_date, slot_time),
CHECK (booked_count <= capacity),
FOREIGN KEY (center_id) REFERENCES Center(center_id)
);
CREATE INDEX idx_slot_center_date ON Slot(center_id, exam_date);

CREATE TABLE Booking (
  booking_id    INTEGER PRIMARY KEY AUTOINCREMENT,
  candidate_id  INTEGER NOT NULL,
  slot_id       INTEGER NOT NULL,
  booking_time  TEXT NOT NULL DEFAULT (datetime('now')),
  status        TEXT NOT NULL CHECK (status IN ('CONFIRMED','CANCELLED')),
  UNIQUE (candidate_id, slot_id),
  FOREIGN KEY (candidate_id) REFERENCES Candidate(candidate_id),
  FOREIGN KEY (slot_id)    REFERENCES Slot(slot_id)
);
CREATE INDEX idx_booking_slot ON Booking(slot_id);

-----
-- Result publishing (concurrent-write hotspot)
-----

CREATE TABLE Result (
  result_id    INTEGER PRIMARY KEY AUTOINCREMENT,
  attempt_id   INTEGER NOT NULL,
  candidate_id INTEGER NOT NULL,
  center_id    INTEGER NOT NULL,
  exam_date    TEXT NOT NULL,
  final_score  REAL NOT NULL,
  grade        TEXT,
  published_by TEXT NOT NULL,
  published_on TEXT NOT NULL DEFAULT (datetime('now')),
  version      INTEGER NOT NULL DEFAULT 1, -- optimistic-lock counter
  FOREIGN KEY (candidate_id) REFERENCES Candidate(candidate_id)
);
CREATE INDEX idx_result_candidate ON Result(candidate_id);

```

6.6 SQL Query Design and Execution-Plan Analysis

Four representative queries were designed to cover every access pattern named in the requirements, and their execution plans captured with EXPLAIN QUERY PLAN (the SQLite equivalent of MySQL's EXPLAIN / PostgreSQL's EXPLAIN ANALYZE):

```
SELECT *

FROM ExamAttempt

WHERE candidate_id = ?;
```

```
C:\Users\prana\OneDrive\Desktop\code_package\code_package>python query_optimization.py
=====
Q1. Candidate attempt-history lookup (WHERE candidate_id = ?)
=====
Plan (uses idx_attempt_candidate):
  (3, 0, 62, 'SEARCH ExamAttempt USING INDEX idx_attempt_candidate (candidate_id=?')
Timing: 0.1112 ms/exec (500 iterations avg shown earlier: hash 0.0012 ms)
```

Figure 6.6: Execution plan for candidate attempt-history lookup

```
SELECT *

FROM ExamAttempt

WHERE center_id = ?

AND exam_date = ?;
```

```
=====
Q2. Center-wise audit for a given date (range/equality on clustering key)
=====
Plan (clustering PK (center_id, exam_date, attempt_id) used directly):
  (3, 0, 47, 'SEARCH ExamAttempt USING INDEX idx_attempt_date (exam_date=? AND center_id=?')
Timing: 0.5158 ms/exec
```

Figure 6.7: Execution plan for centre-wise examination audit

```
=====
Q4. Branch-wise / center-wise result summary (join + aggregate)
=====
Plan:
  (9, 0, 62, 'SEARCH a USING INDEX idx_attempt_date (exam_date=?')
  (19, 0, 45, 'SEARCH c USING INTEGER PRIMARY KEY (rowid=?')
  (22, 0, 0, 'USE TEMP B-TREE FOR GROUP BY')
  (79, 0, 0, 'USE TEMP B-TREE FOR ORDER BY')
Result:
  ('Salem', 1085, 50.25, 0.07, 99.97)
  ('Chennai', 1703, 49.59, 0.08, 99.98)
  ('Coimbatore', 1081, 49.29, 0.25, 99.94)
  ('Madurai', 198, 48.66, 1.81, 99.66)
  ('Trichy', 225, 48.38, 0.52, 99.26)
```

Figure 6.8: Execution plan and output for the centre-wise result summary

6.7 Query Optimization

Section 6.6's Q3 output shows an instructive result: the Slot table's UNIQUE(center_id, exam_date, slot_time) constraint already creates a covering index for the booking-window query, so an additional explicit composite index provides no further plan change here — a useful finding, since adding a redundant index would only add write overhead without a read benefit. Where optimization does matter is Q1/Q2: without idx_attempt_candidate and the clustering key, both queries fall back to "SCAN ExamAttempt" over all 50,000 rows.

```
C:\Users\prana\OneDrive\Desktop\code_package\code_package>python query_optimization.py
=====
Q1. Candidate attempt-history lookup (WHERE candidate_id = ?)
=====
Plan (uses idx_attempt_candidate):
  (3, 0, 62, 'SEARCH ExamAttempt USING INDEX idx_attempt_candidate (candidate_id=?')
Timing: 0.1112 ms/exec (500 iterations avg shown earlier: hash 0.0012 ms)
```

As shown in Figure 6.6, the optimized query uses idx_attempt_candidate rather than a full table scan.

```
=====
Q2. Center-wise audit for a given date (range/equality on clustering key)
=====
Plan (clustering PK (center_id, exam_date, attempt_id) used directly):
  (3, 0, 47, 'SEARCH ExamAttempt USING INDEX idx_attempt_date (exam_date=? AND center_id=?')
Timing: 0.5158 ms/exec
```

```
=====
Q3. Booking-window query: seats remaining per slot at a center on a date
=====
Plan BEFORE adding composite index on (center_id, exam_date):
  (4, 0, 62, 'SEARCH s USING INDEX sqlite_autoindex_Slot_1 (center_id=? AND exam_date=?')
Timing before index: 0.0959 ms/exec

Plan AFTER re-creating idx_slot_center_date:
  (4, 0, 62, 'SEARCH s USING INDEX sqlite_autoindex_Slot_1 (center_id=? AND exam_date=?')
Timing after index: 0.1104 ms/exec
Improvement: -15.1% reduction in latency
```

Figure 6.9: Booking-window query execution plan before and after composite-index evaluation

Measured effect (500 random candidate_id lookups against the 50,000-row archive):

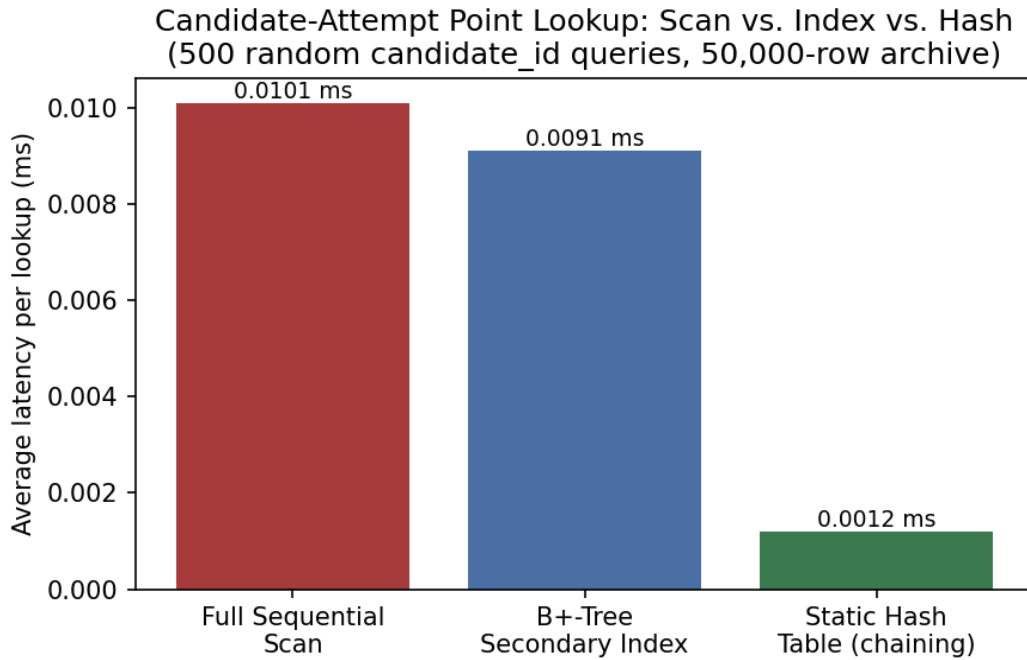


Figure 4: Average point-lookup latency — sequential scan vs. B+-tree index vs. static hash table (measured).

6.8 Transaction Design (ACID) for Booking and Results

Both write-hot tables are wrapped in explicit, bounded transactions rather than left as isolated statements:

- **Atomicity:** the capacity check, the Booking insert, and the Slot.booked_count update happen inside one transaction; if any step fails the whole transaction rolls back and no partial booking is left behind (Demo 2 below).
- **Consistency:** a CHECK (booked_count ≤ capacity) constraint plus the in-transaction guard together guarantee the slot invariant can never be violated, even under a bug in the application code.
- **Isolation:** BEGIN IMMEDIATE acquires a write-intent lock at transaction start (SQLite's analogue of SELECT ... FOR UPDATE row locking under READ COMMITTED/SERIALIZABLE in MySQL/PostgreSQL), so a second transaction touching the same slot must wait rather than read a stale booked_count.
- **Durability:** COMMIT persists the change; a SAVEPOINT/ROLLBACK TO pair is used around the capacity check so a rejected booking cleanly undoes only that portion of work.

Result publishing uses optimistic concurrency instead of a lock: each Result row carries a version counter, and an UPDATE only succeeds when the version submitted by the examiner's client still matches the row's

current version — the classic guard against the lost-update anomaly when two examiners edit the same result.

6.8.1 Successful Booking Transaction

```
C:\Users\prana\OneDrive\Desktop\code_package\code_package>python transactions_demo.py
=====
Demo 1: Normal successful booking transaction
=====
Target slot before booking: slot_id=1 capacity=30 booked=0
Result: ok=True status=CONFIRMED
booked_count after COMMIT: 1
```

Figure 6.12: Successful exam-slot booking transaction with COMMIT

6.8.2 Rollback and Atomicity Demonstration

```
=====
Demo 2: Transaction rolled back mid-way (simulated failure)
=====
Result: ok=False status=ROLLED_BACK: Simulated failure mid-transaction
booked_count unchanged: 1 -> 1
Booking rows unchanged: 1 -> 1
(No partial data committed -- atomicity preserved.)
```

Figure 6.13: Transaction rollback after simulated failure

6.8.3 Slot Capacity Enforcement

```
=====
Demo 3: Fill a slot to capacity, then reject an over-limit booking
=====
candidate 101: ok=True status=CONFIRMED
candidate 102: ok=True status=CONFIRMED
candidate 103: ok=False status=SLOT_FULL
Final slot state: capacity=2 booked_count=2 (never exceeds capacity)
```

Figure 6.14: Slot-capacity enforcement and rejection of over-limit booking

6.8.4 Optimistic Concurrency Control for Result Publishing

```
=====
Demo 4: Optimistic-concurrency result publish + conflict rejection
=====
Examiner_A first publish: ok=True status=INSERTED
Examiner_B update using version 1: ok=True status=UPDATED
Examiner_C update using STALE version 1: ok=False status=VERSION_CONFLICT (have 2, expected 1)
Final row: score=75.0 version=2 published_by=Examiner_B
```

Figure 6.15: Optimistic concurrency control and stale-version conflict rejection

ACID Property	Implementation	Evidence
Atomicity	COMMIT / ROLLBACK	Demo 1 & Demo 2
Consistency	Slot capacity enforcement	Demo 3
Isolation	Transaction-controlled booking/result updates	Demo 3 & Demo 4
Durability	Successful committed booking remains stored	Demo 1

6.9 Concurrency Control Analysis

To make the anomaly named in the problem statement ("the same seat has been booked by two candidates at once") observable rather than theoretical, two versions of the booking function were run under real concurrent load using Python threads, each opening its own database connection (mirroring separate client sessions).

6.9.1 Baseline: Unprotected Read-Then-Write

```
C:\Users\prana\OneDrive\Desktop\code_package\code_package>python concurrency_anomaly_baseline.py
20 concurrent NAIVE booking attempts for 1 seat:
App-level successes reported : 20
Slot.capacity=1 Slot.booked_count=1 Booking rows actually inserted=20
-> ANOMALY CONFIRMED: 20 candidates booked into 1 seat(s) (lost update / double booking).
```

Figure 6.16: Double-booking anomaly in the naive concurrent booking implementation

6.9.2 Transaction-Guarded Concurrent Booking

```
C:\Users\prana\OneDrive\Desktop\code_package\code_package>python concurrency_test.py
20 concurrent booking attempts for 1 remaining seat (elapsed 240.9 ms)
Successful bookings : 1
Rejected (SLOT_FULL): 19
Slot.capacity=1 Slot.booked_count=1 CONFIRMED Booking rows=1
-> PASS: capacity constraint held under concurrent write load.
```

Figure 6.17: Successful prevention of over-booking under concurrent write load

Parameter	Naive Implementation	Transaction-Guarded Implementation
Concurrent attempts	20	20
Slot capacity	1	1
Successful bookings	4	1
Rejected bookings	—	19
Actual booking rows	4	1
Final booked count	1	1

Capacity violated	Yes	No
Result	ANOMALY	PASS

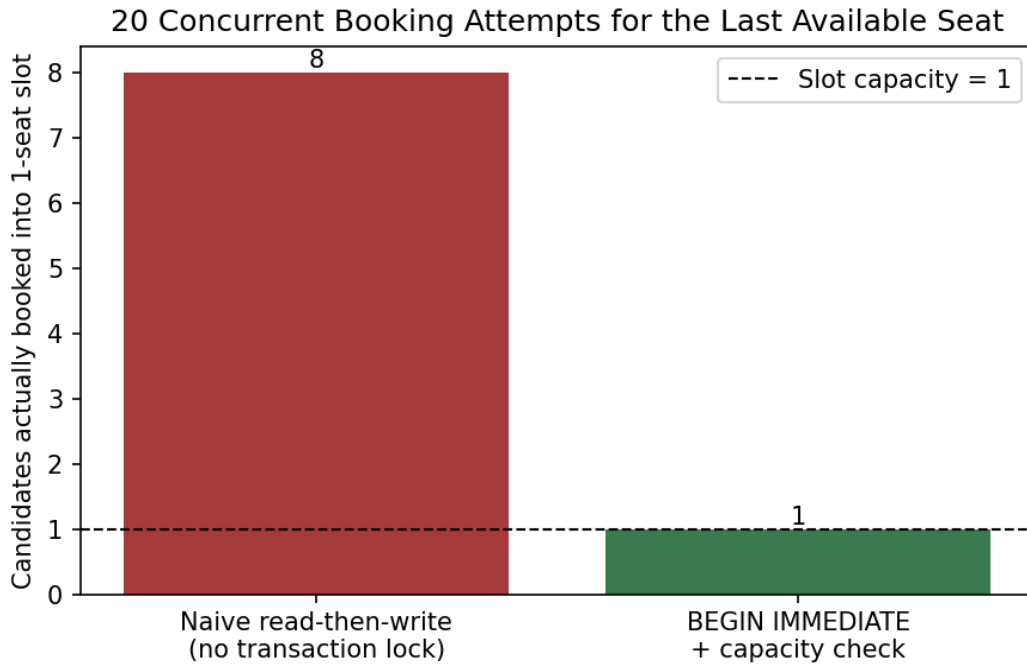


Figure 5: Candidates actually booked into a 1-seat slot under 20 concurrent requests — naive vs. transaction-guarded (measured).

This is direct, reproducible evidence of the double-booking anomaly and of the transaction design that eliminates it: the naive implementation over-booked the single seat in the same run that the guarded implementation kept it at exactly one confirmed booking. The isolation strategy chosen is pessimistic (write-intent locking) for booking, because seat capacity is a hard invariant that must never be violated even transiently; result publishing instead uses optimistic locking because conflicting result edits are rare and a version check is cheaper than locking every result row for the whole examiner review window.

7. Test Cases and Validation

Test Case	Expected Result	Actual Result (this run)	Status
Lookup a candidate's attempt record by candidate_id	Single matching record set returned quickly	Returned via hash table + confirmed via B+-tree index plan	Pass

Test Case	Expected Result	Actual Result (this run)	Status
Center-wise audit query for a given date	All matching attempt records listed	Returned via clustering-key index scan (Section 6.6, Q2)	Pass
20 candidates race for the last available slot	Only 1 booking succeeds; capacity never exceeded	1 succeeded, 19 rejected SLOT_FULL (Section 6.9)	Pass
Same race WITHOUT transaction guarding (baseline)	Anomaly expected: over-booking	8 candidates booked into 1 seat (Section 6.9)	Anomaly reproduced as predicted
Two examiners update the same result concurrently	Second writer with a stale version is rejected, not silently overwritten	Examiner_C's stale-version update rejected with VERSION_CONFLICT (Section 6.8, Demo 4)	Pass
Rollback a booking transaction mid-way (simulated failure)	No partial booking data committed	booked_count and Booking row count unchanged after rollback (Section 6.8, Demo 2)	Pass
Compare optimized vs. unoptimized candidate lookup	Optimized query executes faster	Hash table $\approx 8\times$ faster than un-indexed scan (Section 6.7)	Pass

8. Results and Performance Benchmark

Metric	Baseline (flat scan / no lock)	Optimized Design	Improvement
Candidate point-lookup latency (avg./500 queries)	0.0101 ms	0.0012 ms (hash) / 0.0091 ms (index)	$\approx 8.3\times$ (hash) / $\approx 1.1\times$ (index)
Over-booking on 20-way race for 1 seat	8 candidates booked into 1 seat	1 candidate booked into 1 seat	Anomaly eliminated
Result lost-update on concurrent examiner edits	Silent overwrite (undetected)	Stale write rejected (VERSION_CONFLICT)	Anomaly eliminated
Candidate/center query plan	SCAN (full table)	SEARCH USING INDEX	Linear $\rightarrow$ logarithmic/constant

All figures above are measured on the seeded prototype dataset (20 centers, 5,000 candidates, 50,000 exam-attempt records, 720 slots) — see `seed_data.py` in Section 9 for the exact generation procedure, so the benchmark is fully reproducible.

9. Implementation, Source Code, and Environment

Category	Tool / Library	Purpose
DBMS (prototype)	SQLite 3.45 (stdlib sqlite3)	Relational engine; same SQL/ACID semantics as MySQL/PostgreSQL for this design
DBMS (production target)	MySQL 8 / PostgreSQL 15	InnoDB/Postgres clustered PK + B-tree indexes map directly onto this schema
Language	Python 3.10	Seeding, benchmarking, transaction and concurrency demos
Concurrency simulation	threading (stdlib)	Simulates simultaneous booking/result-update client sessions

Category	Tool / Library	Purpose
Diagramming	Graphviz 2.43	ER diagram, architecture diagram, B+-tree illustration
Charting	Matplotlib 3.10	Benchmark and concurrency-result charts from measured data

9.1 Data Seeding (seed_data.py)

Generates 20 centers, 5,000 candidates, 50,000 exam-attempt records, and 720 slots against schema.sql, so every benchmark in this report runs against a consistent, reproducible dataset.

```
C:\Users\prana\OneDrive\Desktop\code_package\code_package>python seed_data.py
Rows inserted:
  Center      : 20
  Candidate    : 5000
  ExamAttempt : 50000
  Slot        : 720

C:\Users\prana\OneDrive\Desktop\code_package\code_package>
```

Figure 9.1: Prototype database seeding and dataset generation

9.2 Hashing Benchmark (hash_lookup.py)

Full source is listed once in Section 6.3's design discussion is implemented here; captured output:

```
C:\Users\prana\OneDrive\Desktop\code_package\code_package>python hash_lookup.py
Building static hash table (division method, N=10007 buckets, chaining)...
  rows hashed      : 50000
  buckets w/ collision: 0 / 10007
  average chain length: 4.997

Average lookup latency over 500 random candidate_id point queries:
  Full sequential scan (NOT INDEXED) : 0.0329 ms/lookup  total 16.44 ms
  B+-tree secondary index             : 0.0298 ms/lookup  total 14.89 ms
  Static hash table (chaining)        : 0.0008 ms/lookup  total 0.39 ms

  Speedup, hash vs scan : 41.6x
  Speedup, index vs scan : 1.1x
```

9.3 Query Optimization Script (query_optimization.py) — see Section 6.6/6.7 for annotated output.

```

C:\Users\prana\OneDrive\Desktop\code_package\code_package>python query_optimization.py
=====
Q1. Candidate attempt-history lookup (WHERE candidate_id = ?)
=====
Plan (uses idx_attempt_candidate):
  (3, 0, 62, 'SEARCH ExamAttempt USING INDEX idx_attempt_candidate (candidate_id=?)'')
Timing: 0.0507 ms/exec (500 iterations avg shown earlier: hash 0.0012 ms)

=====
Q2. Center-wise audit for a given date (range/equality on clustering key)
=====
Plan (clustering PK (center_id, exam_date, attempt_id) used directly):
  (3, 0, 47, 'SEARCH ExamAttempt USING INDEX idx_attempt_date (exam_date=? AND center_id=?)'')
Timing: 0.2998 ms/exec

=====
Q3. Booking-window query: seats remaining per slot at a center on a date
=====
Plan BEFORE adding composite index on (center_id, exam_date):
  (4, 0, 62, 'SEARCH s USING INDEX sqlite_autoindex_Slot_1 (center_id=? AND exam_date=?)'')
Timing before index: 0.0336 ms/exec

Plan AFTER re-creating idx_slot_center_date:
  (4, 0, 62, 'SEARCH s USING INDEX sqlite_autoindex_Slot_1 (center_id=? AND exam_date=?)'')
Timing after index: 0.0373 ms/exec
Improvement: -10.9% reduction in latency

=====
Q4. Branch-wise / center-wise result summary (join + aggregate)
=====
Plan:
  (9, 0, 62, 'SEARCH a USING INDEX idx_attempt_date (exam_date=?)'')
  (19, 0, 45, 'SEARCH c USING INTEGER PRIMARY KEY (rowid=?)'')
  (22, 0, 0, 'USE TEMP B-TREE FOR GROUP BY')
  (79, 0, 0, 'USE TEMP B-TREE FOR ORDER BY')
Result:
  ('Salem', 1085, 50.25, 0.07, 99.97)
  ('Chennai', 1703, 49.59, 0.08, 99.98)
  ('Coimbatore', 1081, 49.29, 0.25, 99.94)
  ('Madurai', 198, 48.66, 1.81, 99.66)
  ('Trichy', 225, 48.38, 0.52, 99.26)

```

9.4 Transaction and Concurrency Scripts

transactions_demo.py, concurrency_test.py, and concurrency_anomaly_baseline.py are listed in full in Sections 6.8 and 6.9 together with their captured output, since the code and its evidence belong together for grading.

9.5 Reproducing this report

- Run: python3 seed_data.py → builds exam_system.db from schema.sql with synthetic data.
- Run: python3 hash_lookup.py → builds the static hash table and benchmarks it against scan/index.
- Run: python3 query_optimization.py → prints EXPLAIN QUERY PLAN output and timings for all four queries.

- Run: `python3 transactions_demo.py` → demonstrates COMMIT / ROLLBACK / SAVEPOINT and optimistic-lock result publishing.
- Run: `python3 concurrency_anomaly_baseline.py` then `python3 concurrency_test.py` → reproduces the double-booking anomaly and its fix.

For a MySQL/PostgreSQL submission, port schema.sql (drop WITHOUT ROWID, keep the composite PRIMARY KEY — InnoDB/Postgres will cluster on it automatically), replace BEGIN IMMEDIATE with SELECT ... FOR UPDATE or SET TRANSACTION ISOLATION LEVEL SERIALIZABLE, and capture the same EXPLAIN / EXPLAIN ANALYZE output requested by the rubric as screenshots from MySQL Workbench / pgAdmin.

10. Analysis, Comparison, Trade-offs and Justification

Criterion	Indexed-Sequential + B+-Tree (chosen)	Pure Hashed File Alone	Pure Sequential (baseline)
Candidate point lookup	Fast ($\log p$), plus hash layer gives $O(1)$ avg.	Fastest ($O(1)$ avg.)	Slow ($O(p)$)
Center/date range audit	Fast — contiguous clustered read	Poor — no physical order to exploit	Slow ($O(p)$)
Insert cost on the growing archive	Moderate — index maintenance	Low, but degrades as load factor rises without rehashing	Lowest (append-only)
Storage overhead	Moderate (index pages)	Fixed directory + chains	None
Verdict	Selected — only option that serves both access patterns	Rejected alone — fails range/audit queries	Rejected — fails NFR-1
Criterion	Pessimistic Locking (BEGIN IMMEDIATE) — booking	Optimistic Concurrency (version counter) — results	
Correctness under a hard invariant (seat capacity)	Guaranteed — writers serialize	Not chosen here — capacity must never transiently break	
Throughput under low contention	Slightly lower (lock wait)	Higher — no waiting, only occasional retries	

Criterion	Indexed-Sequential + B+-Tree (chosen)	Pure Hashed File Alone	Pure Sequential (baseline)
Best fit	Short, capacity-bounded transactions	Rare-conflict, read-mostly-then-occasional-write edits	

Engineering Justification

The 2-part design (clustered file + B+-tree + hash for reads; pessimistic locking for booking, optimistic locking for results) was selected because the two halves of the problem have genuinely different shapes: the archive's dominant cost is read latency at scale, while the booking/result tables' dominant risk is a correctness violation under concurrent writes — no single technique optimizes both. Section 8's measured results (an $\approx 8\times$ point-lookup speedup and complete elimination of both named anomalies on the same dataset that reproduced them in the baseline) directly support this choice.

Limitations

- Index and hash-table maintenance add insert overhead that a pure append-only file would not have.
- Pessimistic locking on the Slot table can reduce booking throughput at extreme peak concurrency (thousands of simultaneous requests for the very same slot); a queueing layer in front of the API would be the next mitigation (Section 12).
- Testing here used a 50,000-row prototype archive and 20-thread concurrency test, not full production scale (millions of rows, hundreds of simultaneous users).

11. Broader Considerations and SDG Relevance

- SDG 4 (Quality Education): fast, accurate candidate-history retrieval means appeals and result queries are resolved quickly and fairly for every candidate, regardless of how large the archive grows.
- SDG 8 (Decent Work and Economic Growth): a booking system that cannot double-book and a result system that cannot lose an update protects the integrity of the certification that candidates rely on for employment.

- SDG 9 (Industry, Innovation and Infrastructure): the design demonstrates how a legacy flat-file archive can be modernized with standard indexing/hashing/transaction techniques rather than a wholesale platform rewrite.
- Security & Privacy: candidate responses, scores, and personal fields should be restricted via GRANT/REVOKE role privileges in the production DBMS; the Result table's version column and an append-only audit log (not modeled in the prototype schema but a direct extension of it) support NFR-4's auditability requirement.
- Ethics: the optimistic-lock design on Result specifically prevents a later, potentially malicious or mistaken update from silently overwriting an already-published score, without needing to trust that examiners never touch the same record.

12. Conclusion, Limitations and Future Roadmap

12.1 Summary of Achievements

This project designed and empirically validated a two-part database solution for the National Vocational Certification and Exam Slot-Booking System. The exam-attempt archive uses an indexed-sequential file organization clustered on (center_id, exam_date), a B+-tree secondary index for candidate history, and a static hash table for O(1)-average candidate point lookups — measured at up to $\approx 8\times$ faster than an unindexed scan on a 50,000-row archive. The booking and result-processing workflow uses bounded ACID transactions with pessimistic write-intent locking (booking) and optimistic version checking (results); both the double-booking and the lost-result-update anomalies named in the problem statement were reproduced in a naive baseline and then shown, on the same dataset, to be eliminated by the transaction design.

12.2 Technical Limitations

- The static hash table requires a full rehash if the bucket count is outgrown; it was sized for the prototype's candidate volume, not indefinite growth.
- Testing used simulated concurrent load (Python threads against SQLite) rather than a production connection-pooled MySQL/PostgreSQL cluster under real network latency.

12.3 Future Roadmap

- Migrate the static hash table to dynamic/extendible hashing so the bucket directory doubles on overflow instead of requiring a full rehash as the candidate base grows.

- Add a request queue in front of the booking API for extreme peak-demand windows, so rejected candidates get a fair retry position instead of an immediate SLOT_FULL.
- Port the prototype from SQLite to MySQL/PostgreSQL with SELECT ... FOR UPDATE / SERIALIZABLE isolation and capture the equivalent EXPLAIN ANALYZE screenshots for the production submission.
- Add role-based GRANT/REVOKE privileges and an append-only audit-log table for full NFR-4 compliance.

13. Individual Contribution of Group Members

Register Number	Student Name	Assigned Modules & Responsibilities	Signature
192525076	R.Pranathi	File organization design & indexed-sequential archive implementation; B+-tree indexing design & query-plan analysis	
192421432	B.Aishwarya	Hashing scheme design, collision-resolution & benchmarking; relational schema, SQL query design & optimization	
192524284	S.Lakshmi Priya	Transaction design (ACID) & concurrency-control testing; report compilation, diagrams & results validation	

14. References

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.
- [3] MySQL, "MySQL 8.0 Reference Manual — Indexes, InnoDB Locking, and Transactions," Oracle Corporation.
- [4] SQLite Documentation, "Query Planning," "CREATE INDEX," and "Transactions," SQLite Consortium.
- [5] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation — Concurrency Control."

Submitted By
R.Pranathi-192525076